



MICROSOFT

FABRIC

A scenic landscape featuring a calm lake reflecting the surrounding environment. In the foreground, a rustic wooden lakehouse with a dark roof sits on stilts over the water. To the right, a small white church with a dark steeple is nestled among green trees on a hillside. The background is dominated by towering, rugged mountains with patches of snow and mist swirling around their base. The sky is filled with soft, white clouds, and the overall atmosphere is peaceful and majestic.

Bas Land

Your first
Lakehouse
using Microsoft

Data Saturday Gothenburg 2025

Sponsors



The after-party sponsor



NEIGHBOUR
HOOD



Bas Land

- Data consultant since 2013
- I've done Power Pivot, SQL Server, Power BI, Azure SQL, and now Fabric
- Dataplatform MVP, since last week 😊
- Brazilian jiu-jitsu purple belt, weight lifting, running
- Husband and father



Connect with me

LinkedIn



YouTube



Blog: ThatFabricGuy.com



Today

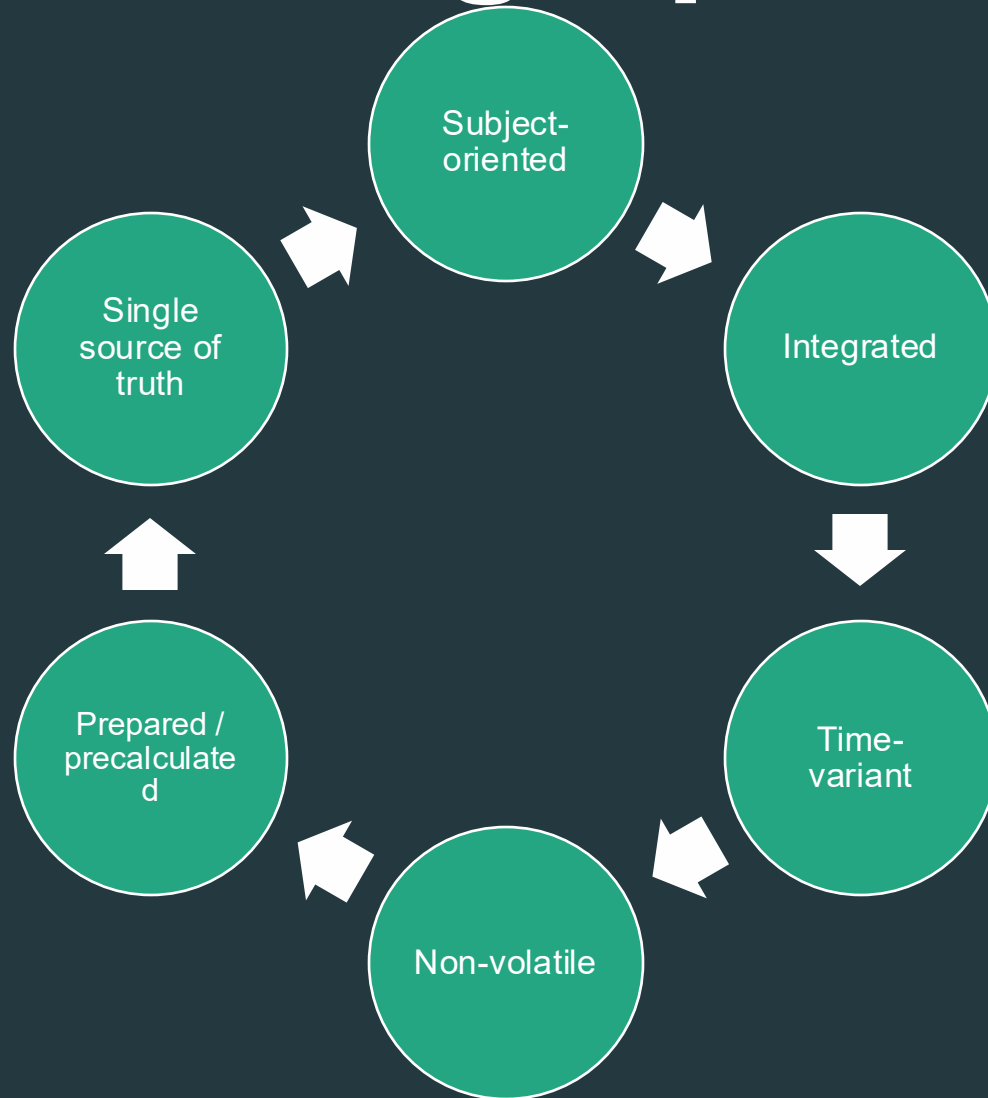
- Core concepts for data warehousing in Fabric
- Your first Fabric Lakehouse:
 - Rulebook
 - ELT process
 - End-to-end solution design with examples
- Takeaways



Core concepts



General design principles



Lakehouse: under the hood



Lakehouse: under the hood

Ingest



Pipelines
Notebooks
Dataflows

Store



OneLake
Data lake blob storage

Process



Notebooks
(PySpark)



Fabric concepts

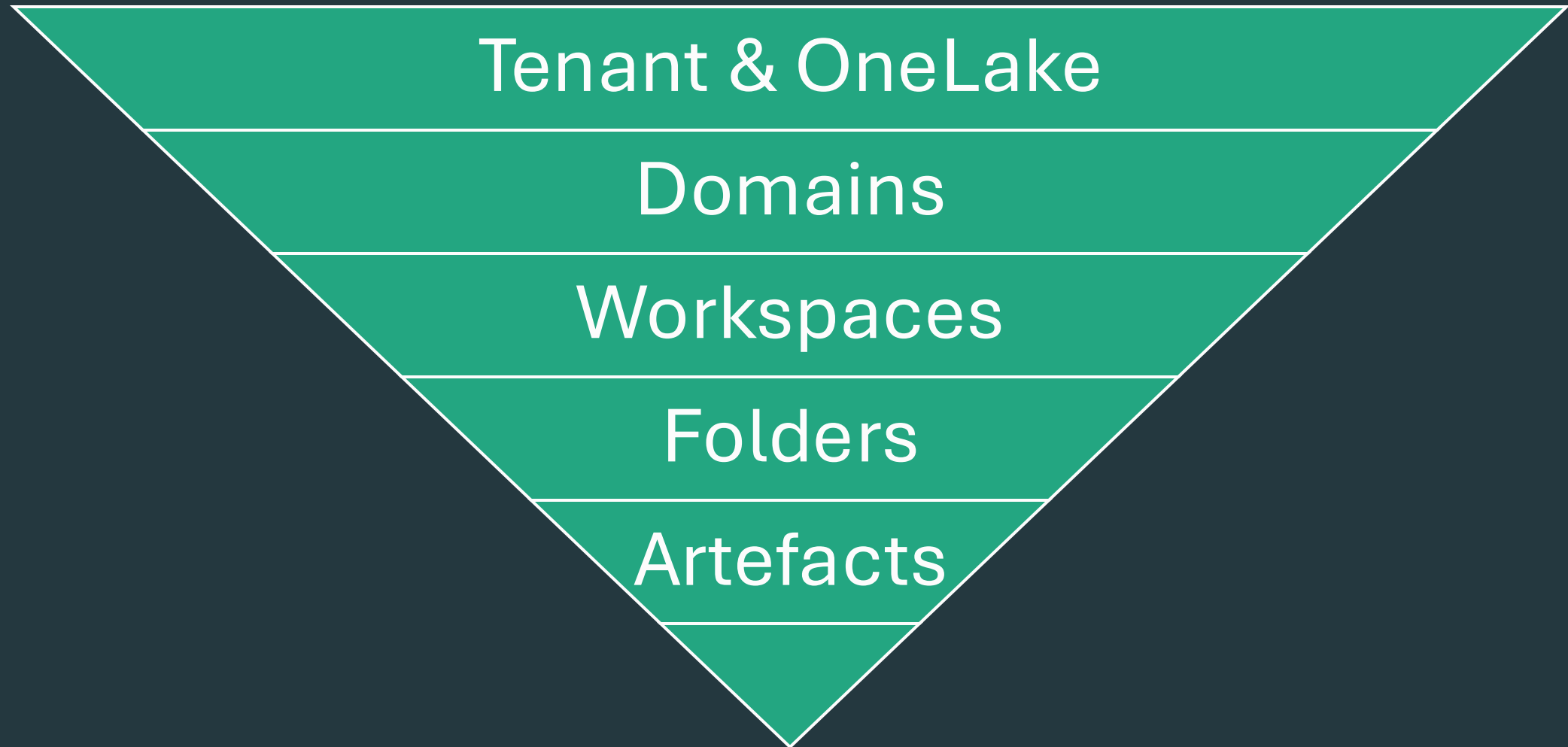
1. Tenant design

2. Storage

3. Compute



Fabric: tenant design



Fabric: tenant design

Fabric Tenant

Domain with workspace with DTAP environments



Bronze DEV



Silver DEV



Gold DEV



Sandbox intern



Bronze ACC



Silver ACC



Gold ACC



Sandbox machine learning



Bronze PRD



Silver PRD



Gold PRD



Sandbox ...

Domain 'Board Reporting'



Gold Board DEV



Gold Board ACC

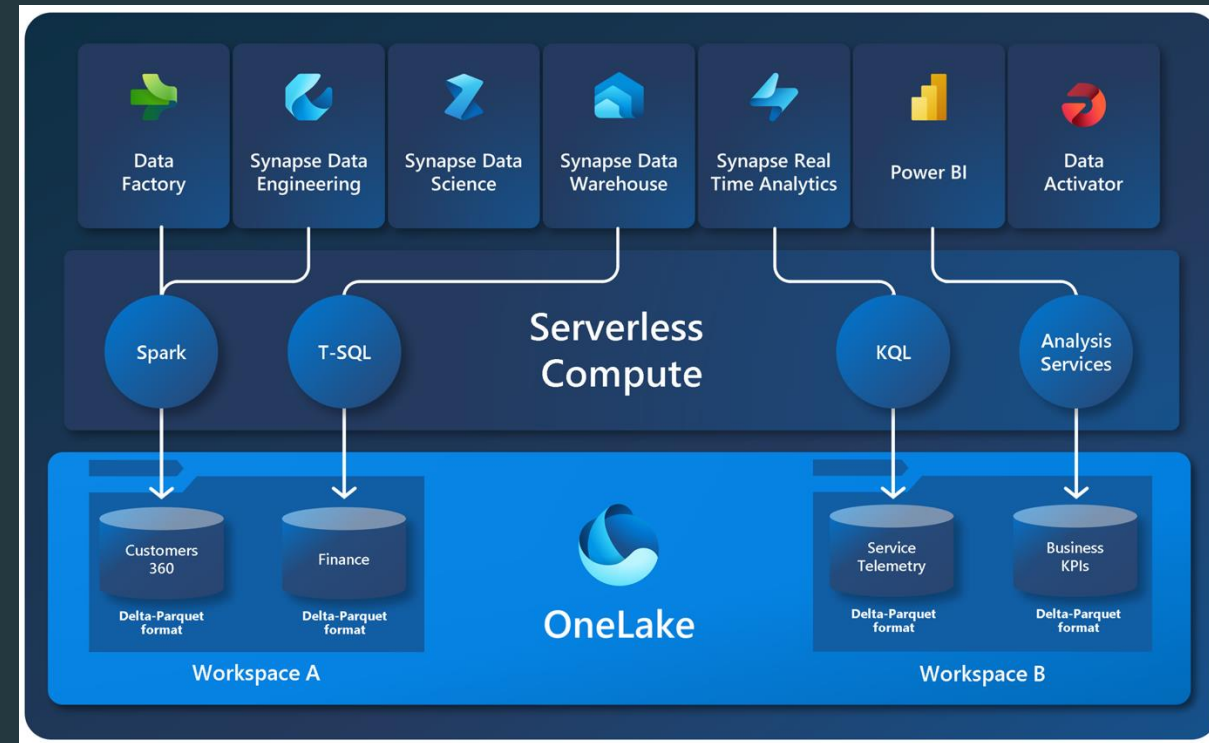


Gold Board PRD



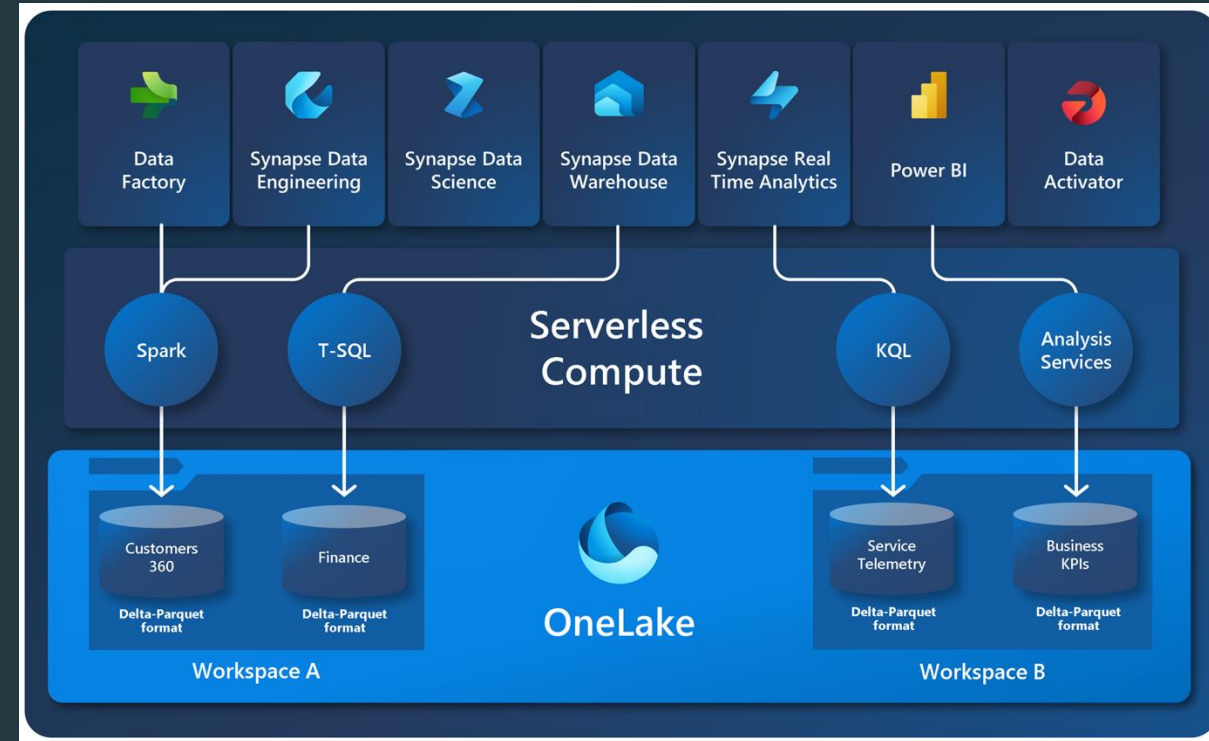
Fabric: storage

- Storage in Fabric is called 'OneLake'
- It's blob storage, one OneLake per organisation
- Storage is cheap! $\pm$ 2,2 cents per GB per month



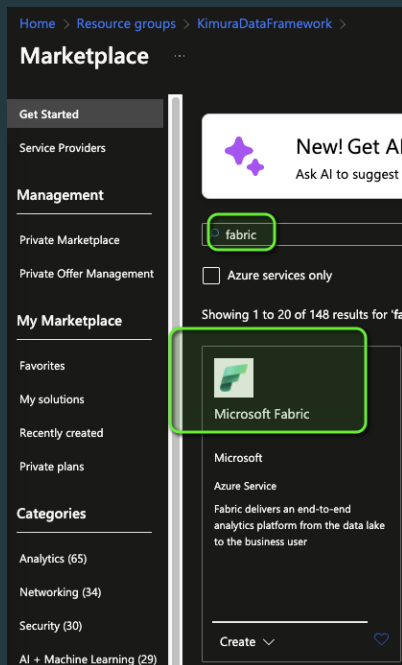
Fabric: compute

- Compute in Fabric is Serverless, included in your SaaS capacity
- Preconfigured Spark cluster for compute
- Compute = expensive. Be smart about it 😊



Buying the Fabric compute capacity

- portal.azure.com
- I'd create at least a new Resource Group, maybe even a Subscription (depending on organisation requirements)



Basics Tags Review + create

Create Fabric capacity that you can use with your Fabric workspaces.

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all of your resources.

Subscription * ⓘ Kimura Default Subscription (MPN) ▼

Resource group * ⓘ KimuraDataFramework ▼

[Create new](#)

Capacity details

Name your Capacity and select a location.

Capacity name * ⓘ kdffabriccapacity ✓

Region * West Europe ▼

Size ⓘ **F64**
64 Capacity units
[Change size](#)

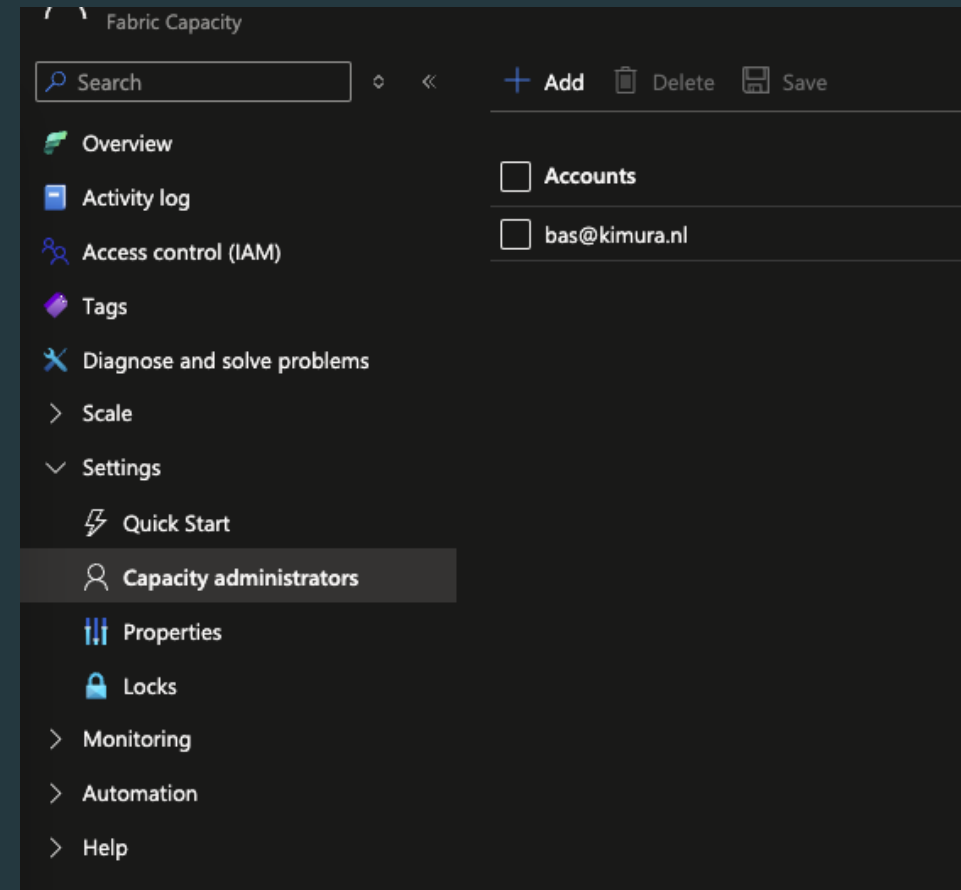
Fabric capacity administrator * ⓘ bas@kimura.nl ✓
[Select](#)

Select the resource size		
SKU	Capacity Units	COST (ESTIMATED/MONTH)
F2	2	€300.66
F4	4	€601.32
F8	8	€1,202.65
F16	16	€2,405.30
F32	32	€4,810.59
F64	64	€9,621.19
F128	128	€19,242.38
F256	256	€38,484.76
F512	512	€76,969.52
F1024	1024	€153,939.04
F2048	2048	€307,878.07



Buying the Fabric compute capacity

- After deployment, configure the Capacity Administrator:
- Unfortunately, no Entra ID groups...
- And no guest users...
- Yes, I have some beef with MS about this 😊



Creating the first Fabric Workspace

- Head over to app.powerbi.com (or app.fabric.microsoft.com)
- Log in as Capacity Administrator
- Create a new workspace
- Now, pay attention to the License Type

Create a workspace

☐ Pro

Select Pro to use basic Power BI features and collaborate on reports, dashboards, and scorecards. To access a Pro workspace, users need Pro per-user licenses. [Learn more](#)

☐ Trial

Select the free trial per-user license to try all the new features and experiences in Microsoft Fabric for 60 days. A Microsoft Fabric trial license allows users to create Microsoft Fabric items and collaborate with others in a Microsoft Fabric trial capacity. Explore new capabilities in Power BI, Data Factory, Data Engineering, and Real-Time Intelligence, among others. [Learn more](#)

☐ Premium per-user

Select Premium per-user to collaborate using Power BI Premium features, including paginated reports, dataflows, and datamarts. To collaborate and share content in a Premium per-user workspace, users need Premium per-user licenses. [Learn more](#)

☐ Premium capacity

Select premium capacity if the workspace will be hosted in a premium capacity. When you share, collaborate on, and distribute Power BI and Microsoft Fabric content, users in the viewer role can access this content without needing a Pro or Premium per-user license. [Learn more](#)

☐ Embedded

Select embedded if the workspace will be hosted in an Azure embedded capacity. ISVs and developers use Power BI Embedded to embed visuals and analytics in their applications. [Learn more](#)

☒ Fabric capacity

Select Fabric capacity if the workspace will be hosted in a Microsoft Fabric capacity. With Fabric capacities, users can create Microsoft Fabric items and collaborate with others using Fabric features and experiences. Explore new capabilities in Power BI, Data Factory, Data Engineering, and Real-Time Intelligence, among others. [Learn more](#)

Semantic model storage format

☒ Small semantic model storage format

☐ Large semantic model storage format

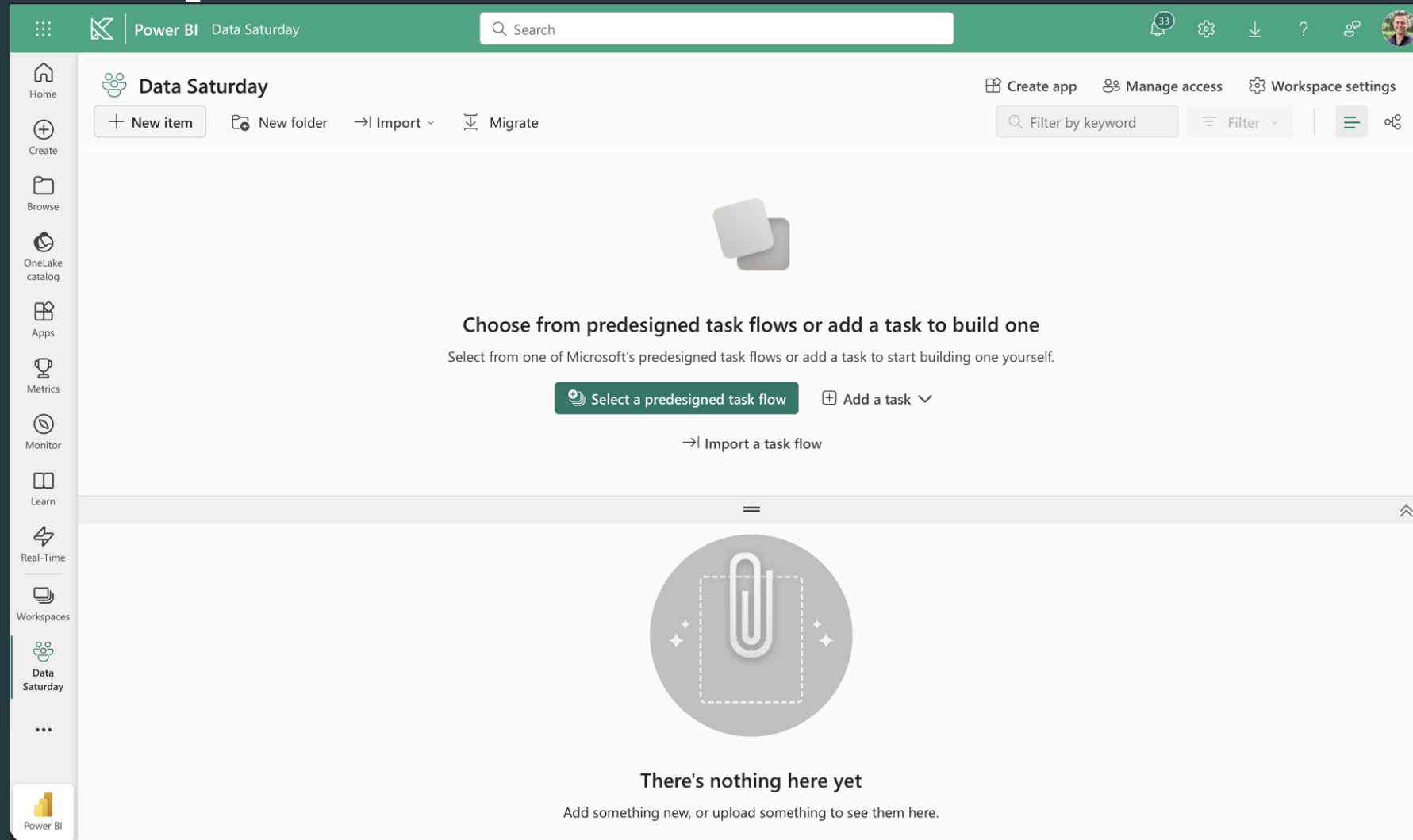
[Learn more about semantic model storage formats](#)

Capacity *

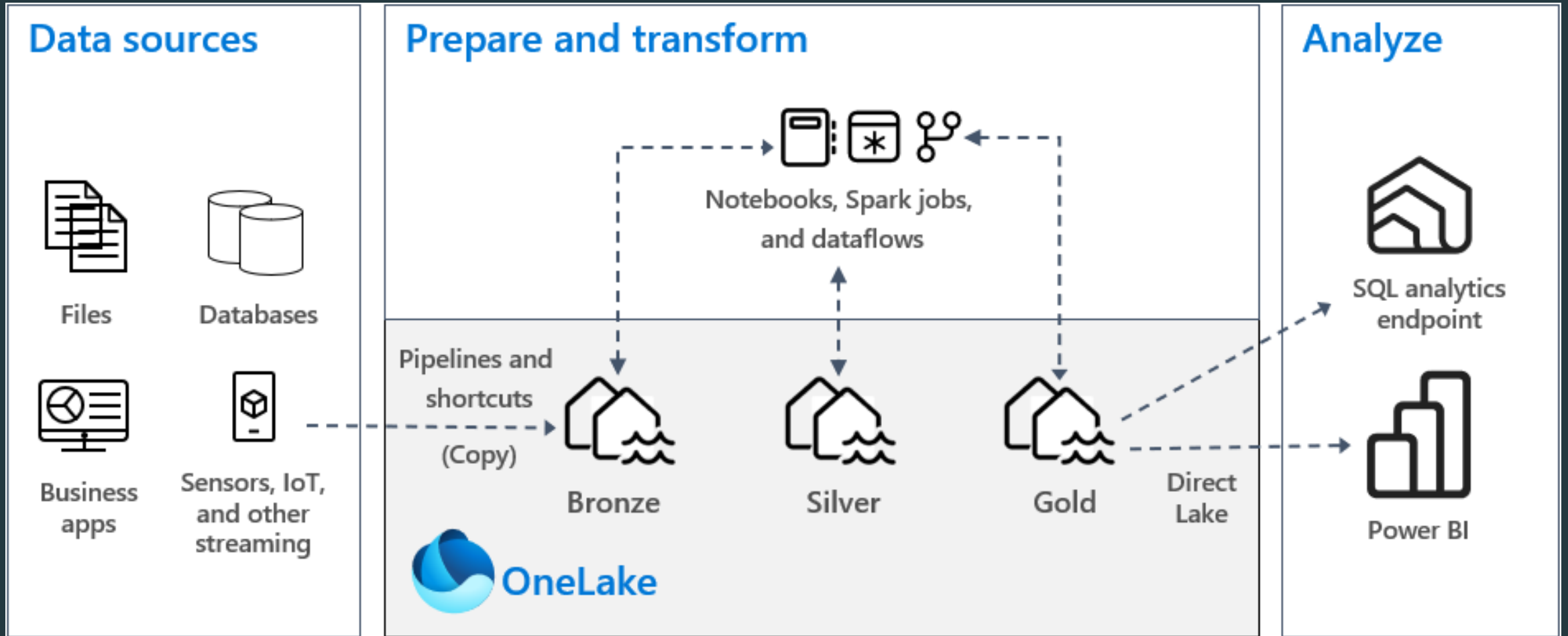
kimuradataframeworkfabriccapacity - West Europe



Creating the first Fabric Workspace

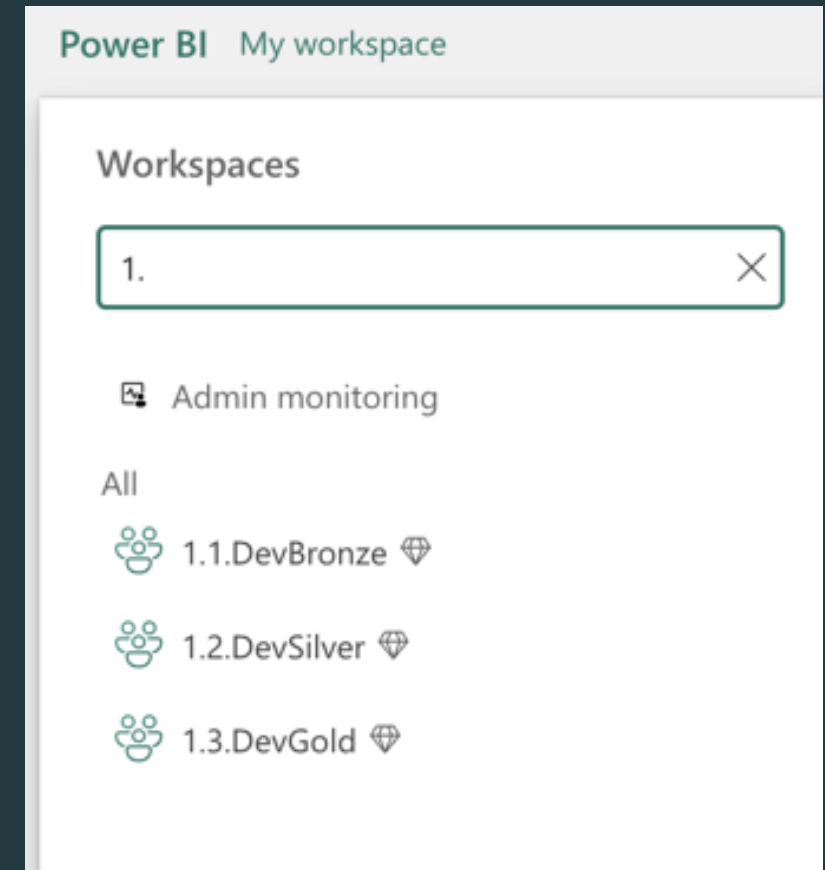


Medallion architecture



Workspaces and Medallion

- One workspace per Medallion stage
- One workspace per DTAP environment
- Workspaces for access management



Your first Fabric Lakehouse



The scenario

- Client: medium-sized consulting company
- Main software: Odoo (ERP), Moneybird (Accounting)
- Data domains: Finance, Projects & Timesheets



The rulebook

- Fabric Pipelines orchestration
- PySpark notebooks for processing
- Reusable code for everything
- Lakehouse for storage



Prerequisites

Setting up:

- Fabric compute capacity (done)
- Fabric workspaces (done)
- Configuring Spark (we skip that for today)
- Setting up the Environment (important!)



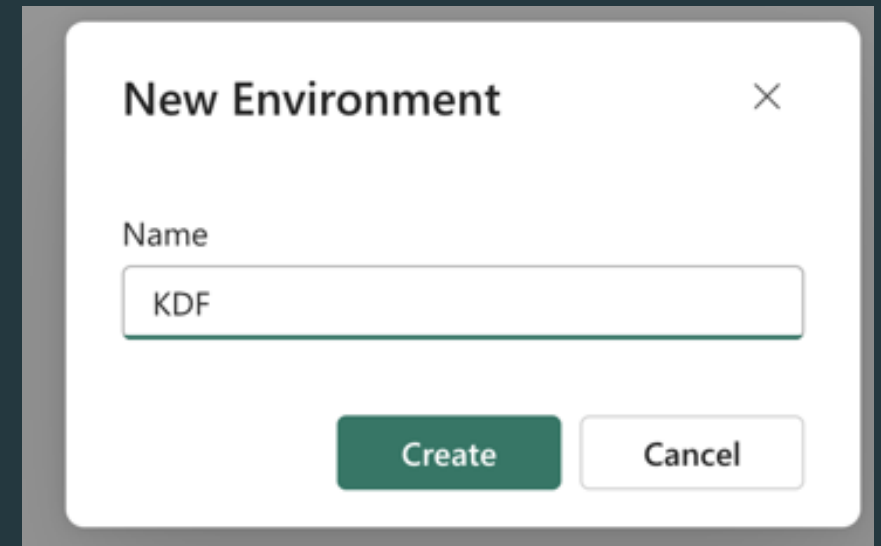
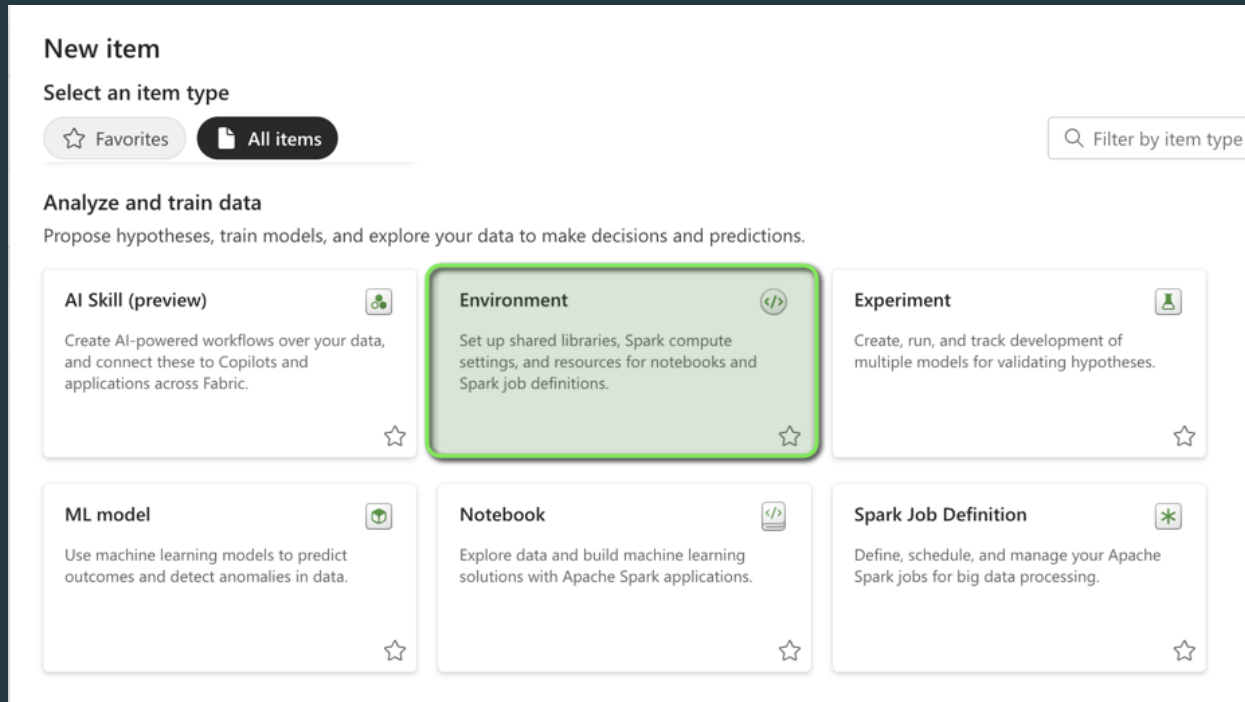
Environments in Fabric

- Fabric contains a lot of Python packages natively
- You can also add packages using the PyPI (Python Package Index)
- And, you can write and include your own code
- This is done using Environments



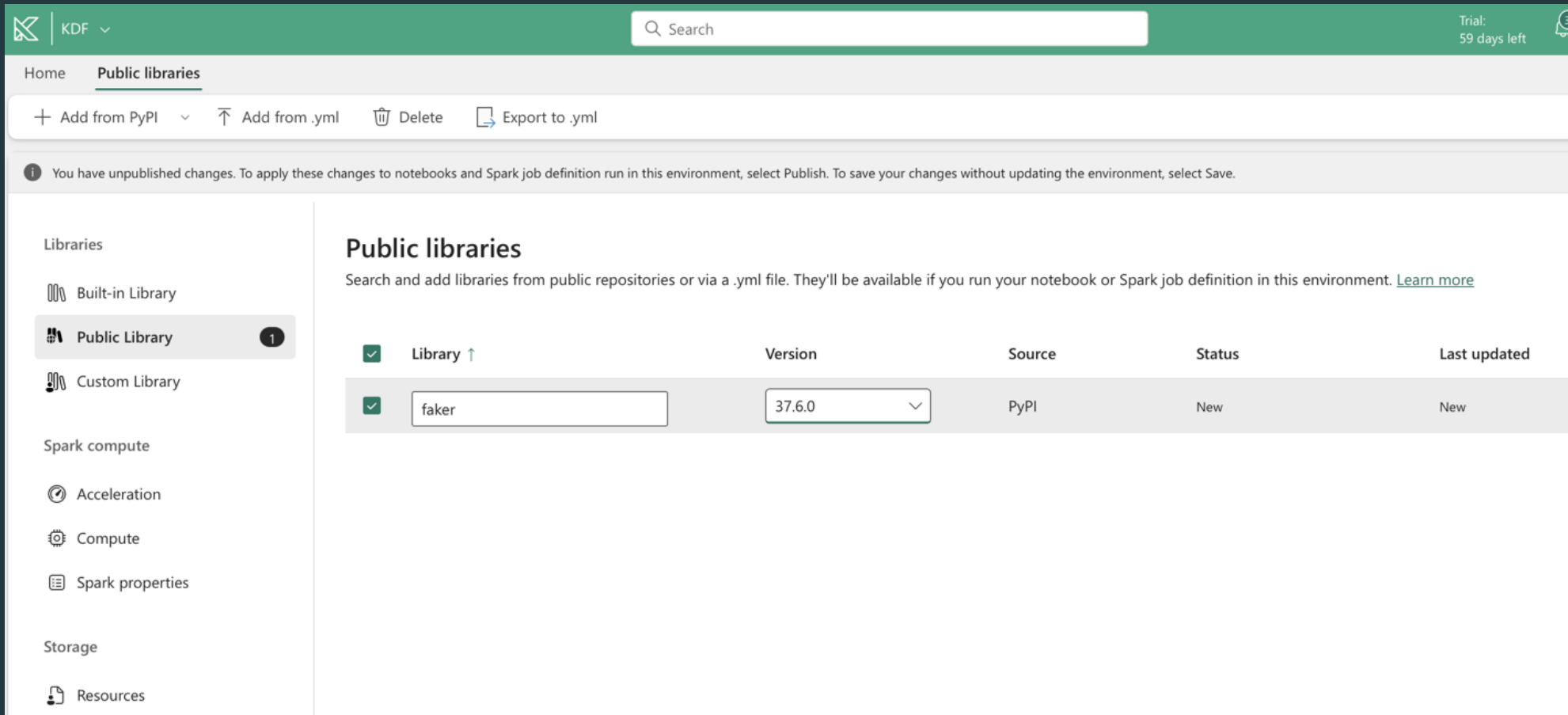
Environments in Fabric

- Create a new environment:



Environments in Fabric

- Add code from PyPI:



The screenshot shows the Databricks Fabric interface for managing public libraries. The top navigation bar is green with the Databricks logo, 'KDF', a search bar, and a trial status indicator ('Trial: 59 days left'). Below the navigation bar, there's a breadcrumb trail 'Home > Public libraries'. A toolbar contains buttons for '+ Add from PyPI', 'Add from .yaml', 'Delete', and 'Export to .yaml'. A notification banner states: 'You have unpublished changes. To apply these changes to notebooks and Spark job definition run in this environment, select Publish. To save your changes without updating the environment, select Save.'

The main content area is titled 'Public libraries' and includes a descriptive text: 'Search and add libraries from public repositories or via a .yaml file. They'll be available if you run your notebook or Spark job definition in this environment. [Learn more](#)'. Below this is a table with the following columns: Library, Version, Source, Status, and Last updated.

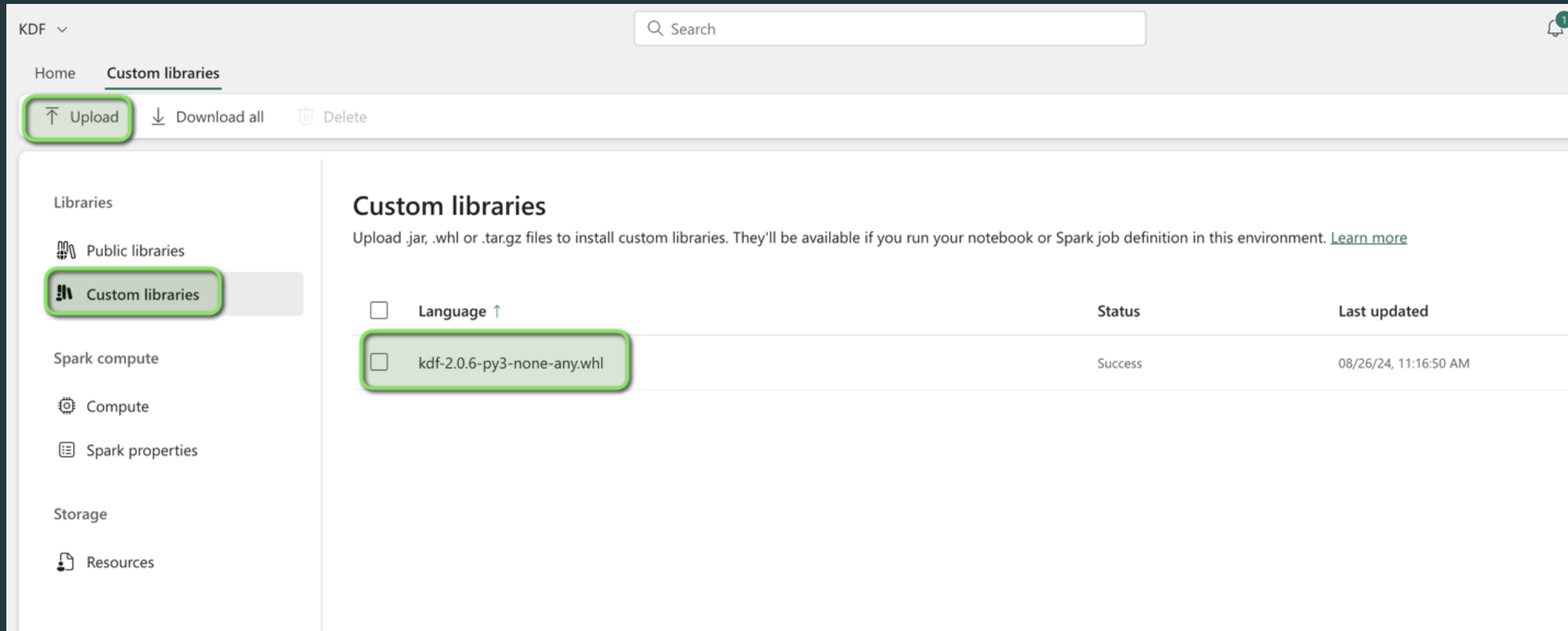
Library	Version	Source	Status	Last updated
☒ faker	37.6.0	PyPI	New	New

The left sidebar contains a 'Libraries' section with 'Built-in Library' and 'Public Library' (which has a '1' badge). Below this are 'Spark compute' options (Acceleration, Compute, Spark properties) and 'Storage' options (Resources).



Environments in Fabric

- Add your own code:



The screenshot shows the Databricks Custom Libraries interface. The top navigation bar includes a search bar and a notification icon. The left sidebar contains a menu with 'Libraries' (Public libraries, Custom libraries), 'Spark compute' (Compute, Spark properties), and 'Storage' (Resources). The main content area is titled 'Custom libraries' and includes an instruction to upload .jar, .whl, or .tar.gz files. Below this is a table with columns for checkboxes, Language, Status, and Last updated. One entry is visible: 'kdf-2.0.6-py3-none-any.whl' with a status of 'Success' and a last updated time of '08/26/24, 11:16:50 AM'. Several elements are highlighted with green boxes: the 'Upload' button, the 'Custom libraries' menu item, the checkbox for the first table entry, and the first table row itself.

KDF ▾

Home Custom libraries

↑ Upload ↓ Download all 🗑 Delete

Libraries

- Public libraries
- Custom libraries**

Spark compute

- Compute
- Spark properties

Storage

- Resources

Custom libraries

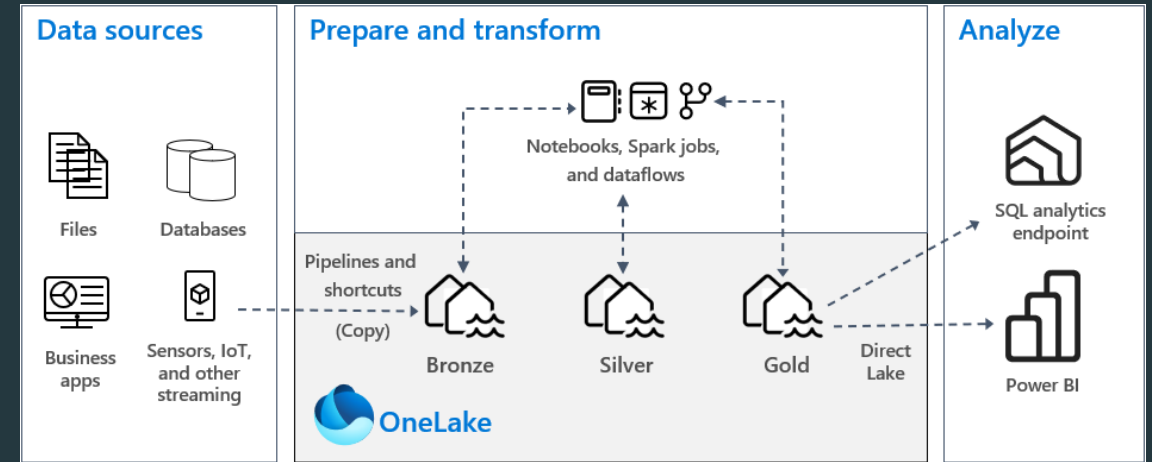
Upload .jar, .whl or .tar.gz files to install custom libraries. They'll be available if you run your notebook or Spark job definition in this environment. [Learn more](#)

☐	Language ↑	Status	Last updated
☐	kdf-2.0.6-py3-none-any.whl	Success	08/26/24, 11:16:50 AM



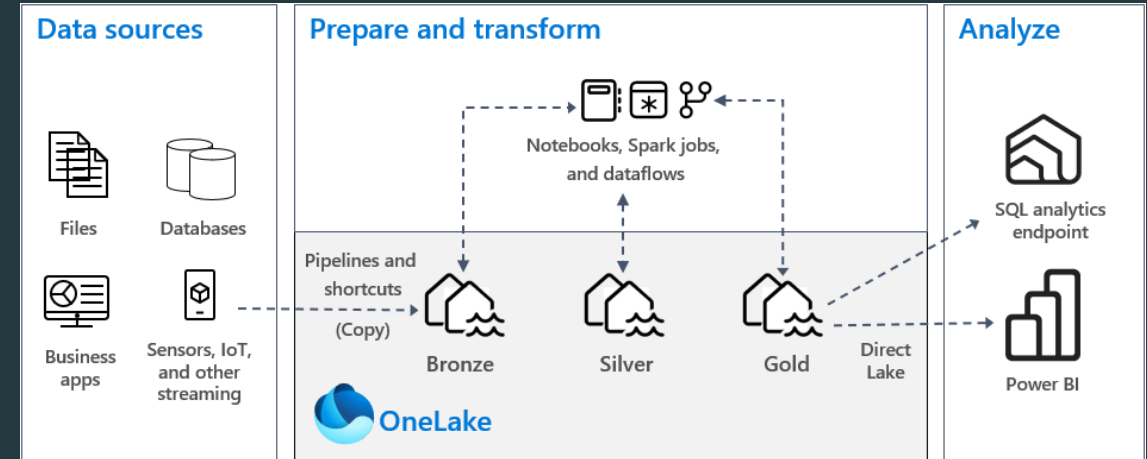
Three lakehouses

- Bronze: timestamped folders, (incremental loads), raw data (files; json, xml, csv, parquet etc)
- Silver: historical archive, normalised schemas, delta tables
- Gold: prepared and modeled DWH tables, dims and facts, delta tables

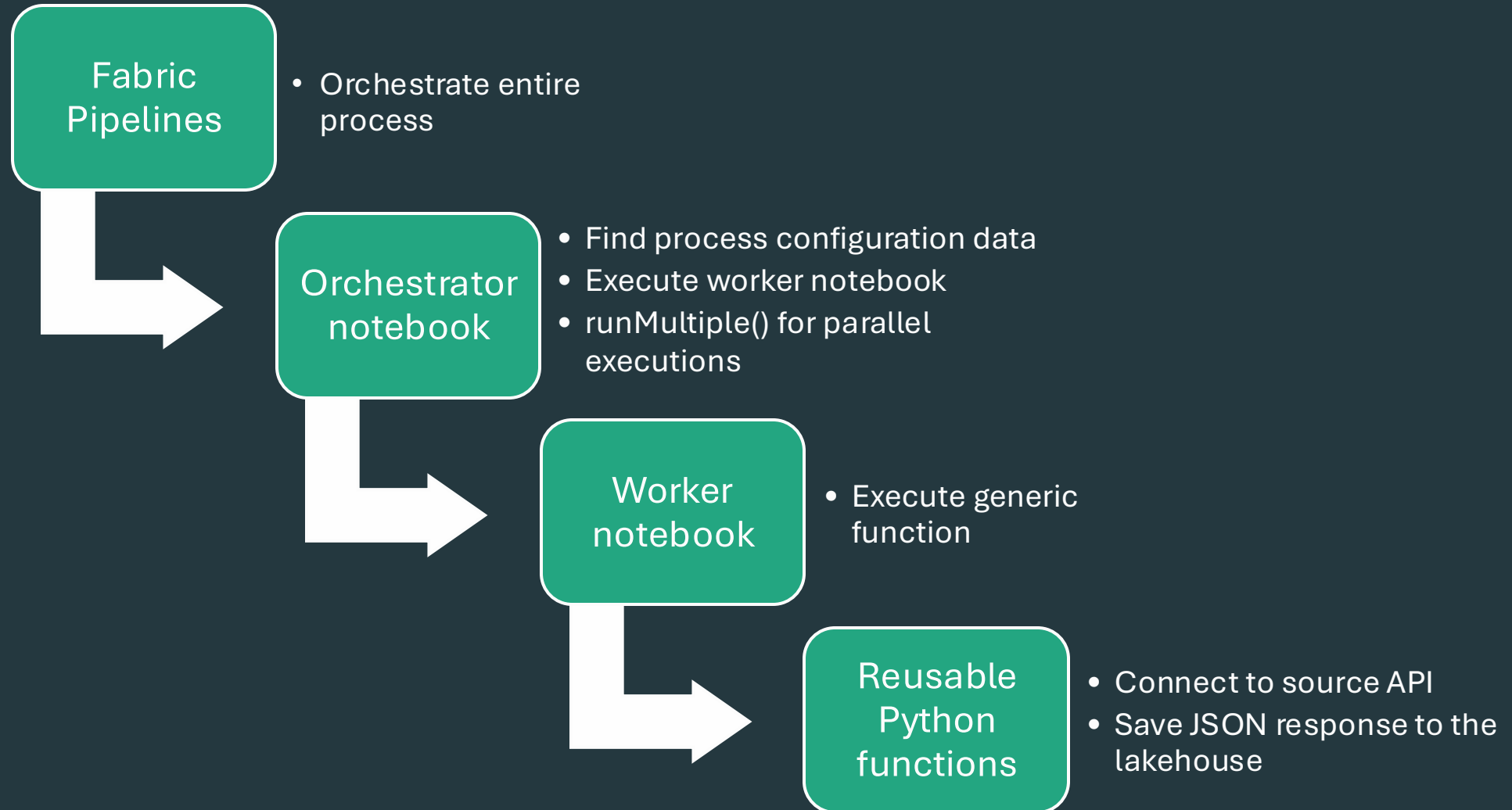


Setting up the ELT process

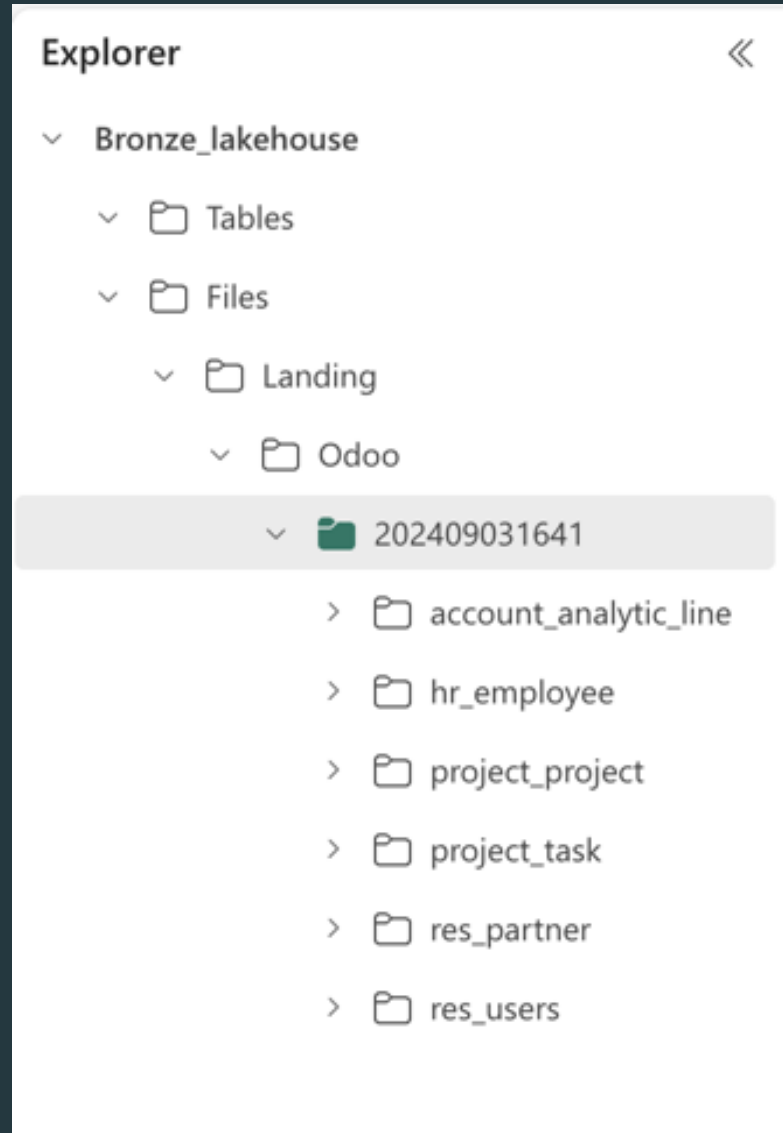
- Keep in mind the Medallion architecture
- Start connecting to data, to extract ERP and Accounting to Bronze
- We do so using Python, executed in Fabric Notebooks, scheduled and run in Fabric Pipelines



ELT: Extract



ELT: Bronze folder structure



- Root folder 'Landing' (for shortcuts!)
- Source system
- Timestamp
- Folder per table/endpoint
- Files managed by Fabric!



Silver lakehouse

Explorer

▼ Silver_lakehouse

▼ Tables

- >  silver_0_ExtractLogging
- >  silver_account_analytic_line

▼ Files

- >  Landing

silver_account_analytic_line

	ABC account_id	0/1 allow_billable	12 amount	0/1 auto_accoun...	ABC category	0/1 code
1	[1,"Internal"]	False	-505.76	False	other	False
2	[1,"Internal"]	False	0	False	other	False
3	[1,"Internal"]	False	-505.76	False	other	False
4	[1,"Internal"]	False	0	False	other	False
5	[1,"Internal"]	False	-505.76	False	other	False
6	[1,"Internal"]	False	0	False	other	False
7	[1,"Internal"]	False	0	False	other	False
8	[1,"Internal"]	False	0	False	other	False
9	[1,"Internal"]	False	0	False	other	False
10	[1,"Internal"]	False	0	False	other	False
11	[1,"Internal"]	False	0	False	other	False
12	[1,"Internal"]	False	0	False	other	False
13	[1,"Internal"]	False	0	False	other	False
14	[159,"V/2024/00...	True	-18.42	False	other	False
15	[149,"V/2024/00...	True	-36.84	False	other	False
16	[164,"Internal"]	False	-189.66	False	other	False
17	[1,"Internal"]	False	-71.88	False	other	False
18	[142,"Sales & Bu...	False	-63.22	False	other	False
19	[149,"V/2024/00...	True	-64.47	False	other	False



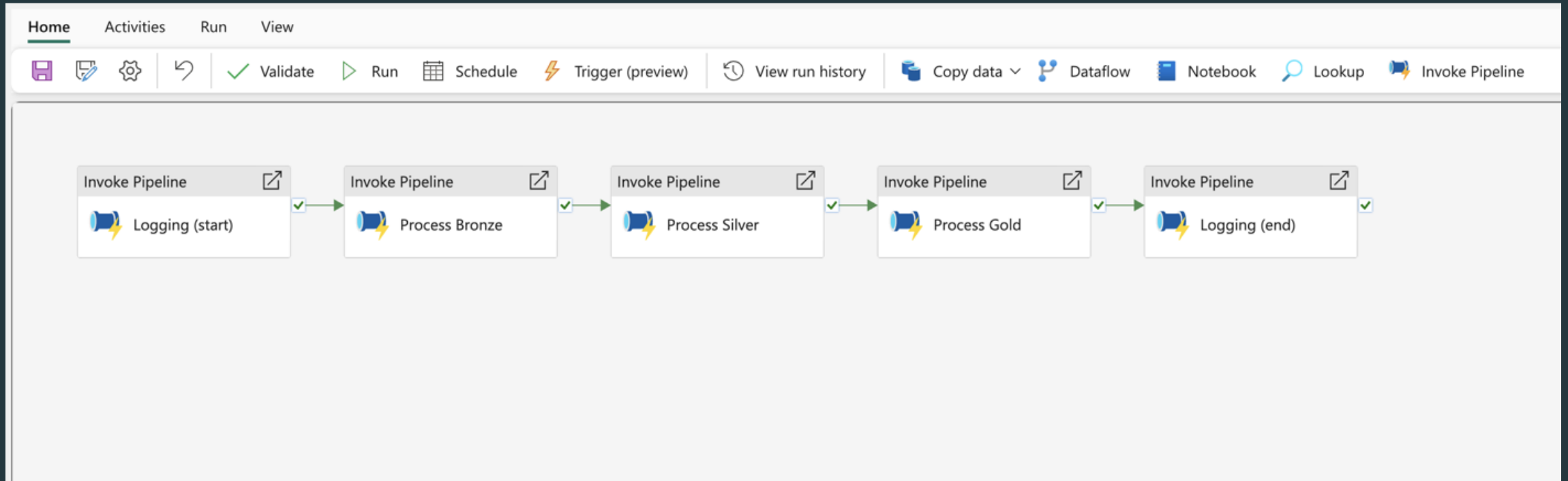
Gold lakehouse

Explorer		dimCustomers				
Gold						
Tables						
dimCustomers						
dimDates						
dimProjects						
dimServices						

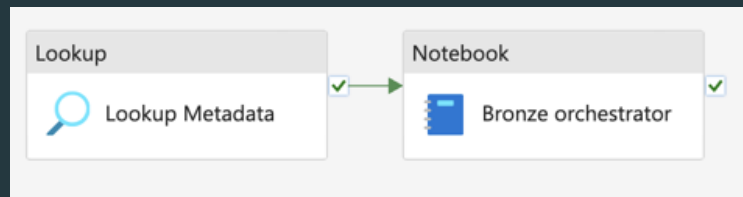
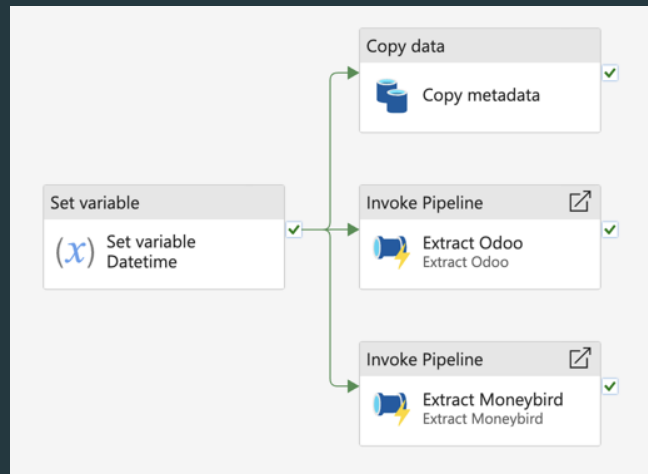
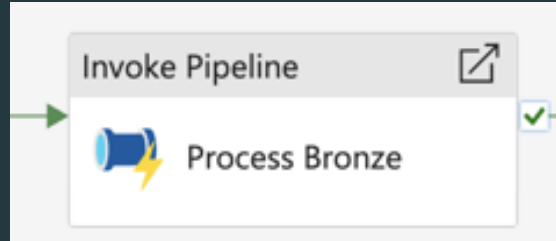
	123	CustomerId	ABC	RelationType	ABC	CustomerNa...	ABC	IsActive	ABC	PaymentType
1	27			Klant				True		banktransfer
2	37			Klant af				True		directdebit
3	103			Klant				True		banktransfer
4	104			Klant				True		banktransfer
5	128			Klant af				True		directdebit
6	130			Klant				True		banktransfer
7	138			Klant				True		directdebit



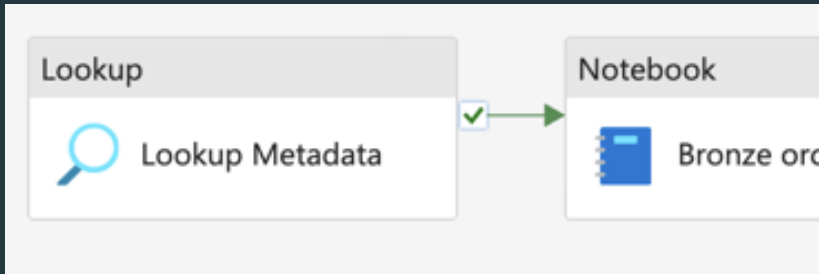
End-to-end solution with examples



End-to-end solution with examples



End-to-end solution with examples



Initial loads

```
1 EXTRACT_TIMESTAMPS_PATH = 'Tables/cfg/odoo_etl_extracttimes'
2 EXTRACT_TABLES_PATH = 'Tables/cfg/odoo_etl_extracttables'
```

[4] ✓ <1 sec - Command executed in 320 ms by Bas Land | Kimura on 8:59:19 AM, 9/06/25

```
1 extract_timestamps_df = spark.read.format('delta').load(EXTRACT_TIMESTAMPS_PATH)
2 last_extract_timestamps_df = extract_timestamps_df.groupBy('source_table_name').max('timestamp').rdd.map(lambda row: (row['source_table_name'], row['timestamp'])).toDF()
3 last_extract_timestamps_df = last_extract_timestamps_df.withColumnRenamed('timestamp', 'last_timestamp')
```

[5] ✓ 1 sec - Command executed in 947 ms by Bas Land | Kimura on 8:59:20 AM, 9/06/25

```
1 extract_tables_df = spark.read.format('delta').load(EXTRACT_TABLES_PATH)
```

[6] ✓ <1 sec - Command executed in 880 ms by Bas Land | Kimura on 8:59:21 AM, 9/06/25

Spark jobs (2 of 2 succeeded) Resources

Set up activities

```
1 activities = []
2
3 for extract_table in extract_tables_df.collect():
4     last_timestamp_row = last_extract_timestamps_df.filter(col('source_table_name') == extract_table['SourceTable']).first()
5
6     if last_timestamp_row:
7         last_timestamp = str(last_timestamp_row['timestamp'])
8     else:
9         last_timestamp = '19000101000000'
10
11     activities.append(
12         {
13             'name': f'{extract_table["SourceTable"]}',
14             'path': 'Worker - Odoo',
15             'timeoutPerCellInSeconds': 9000,
16             'args': {
17                 'timestamp': last_timestamp,
18                 'source_table_name': extract_table['SourceTable'],
19                 'target_table_name': extract_table['TargetTable'],
20                 'columns': extract_table['ColumnList'],
21                 'column_filter': extract_table['ColumnFilterType'],
22                 'table_keys': extract_table['TableKey'],
23                 'incremental_field_name': extract_table['IncrementalField'],
24                 'incremental_field_value': last_timestamp,
25                 'page_size': extract_table['PageSize']
26             }
27         }
28     )
29
30 DAG = {
31     'activities': activities,
32     'timeoutInSeconds': 43200,
33     'concurrency': 3
34 }
35
36 notebookutils.notebook.runMultiple(DAG)
```

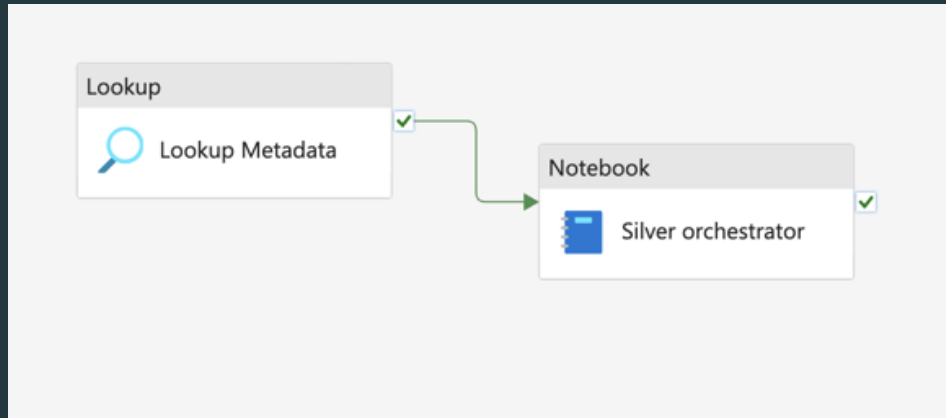
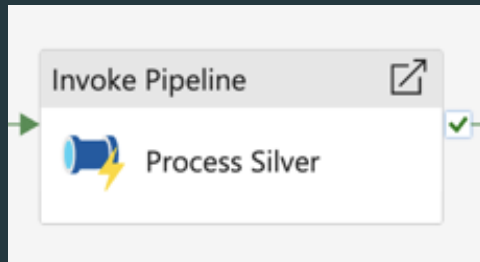
[*] * 56 min 44 sec - Running

Spark jobs (20 of 20 succeeded) Resources Log

PySpark (Python) ...



End-to-end solution with examples



```

1  #Mapping logic
2  gold_table = "dimServices"
3  business_key_columns = ['ServiceId']
4
5  df_mapping = spark.sql("""
6  select
7      t.`data.id` as ServiceId
8      ,t.`data.project_id` as ProjectId
9      ,t.`data.name` as ServiceName
10     ,t.`data.invoice_method` as InvoiceMethod
11     ,t.`data.amount` as Amount
12     ,t.`data.start_date` as StartDate
13     ,t.`data.end_date` as EndDate
14     ,t.`data.revenue_group.label` as RevenueGroup
15     ,t.`data.subscription_cycle` as InvoiceCycle
16     ,t.`data.price` as Price
17     ,t.`data.amount` * t.`data.price` as Value
18     ,case when t.`data.revenue_group.id` = 'revenuegroup:ba06e5bec6c1b6b5' then 'Licenses' else 'Other' end as MRRLabel
19     ,c.Sys_ID as FK_dimCustomers_Sys_ID
20 from Bronze.bronze_Odoo_ProjectsService t
21 left join Bronze.bronze_Odoo_ProjectsProject p
22     on t.`data.project_id` = p.`data.id`
23 left join Silver.silver_dimCustomers c
24     on p.`data.organization.id` = c.OdooId
25 where 1=1
26 """)
27
28 k.gold_load_table( \
29     gold_table = gold_table_name, \
30     df_mapping = df_mapping, \
31     business_key_columns = business_key_columns, \
32     spark_session = spark\
33 )

```

Recap



Recap

1. Keep core-concepts in mind
2. Setup a rulebook and abide by it
3. Use tools and reusable code!
4. Fabric lakehouses aren't that scary 😊



If you need help...

- Contact me please
- My company offers standardised Fabric implementations
- All functions I talked about (and more) available **out of the box**
- And fully customisable



Q&A

Any questions?

